Aula 5: Quando os Objetos Conversam - Relacionamentos (Associação)



Projeto: FiapRide (Sistema de Mobilidade Urbana) + Seu Projeto Pessoal

Foco: Associação de Objetos, Composição e Clean Code

🎯 Objetivo da Aula

Até agora, nós criamos classes como `Passageiro` e `Veiculo`. Elas funcionam perfeitamente, mas no mundo real, objetos não vivem isolados. Uma corrida de aplicativo não existe no vácuo: ela precisa de um Passageiro que a solicitou e de um Veículo que vai realizá-la.

Hoje, você vai aprender a técnica de **Associação** (Relacionamento "Tem-Um"). Vamos construir a classe `Viagem`, que em vez de ter atributos simples como `String` ou `double`, terá **outros objetos** dentro dela!

Conceitos Chave:

- **Associação:** Quando uma classe "tem um" atributo que é do tipo de outra classe.
- **Composição de Objetos:** Construir objetos complexos usando objetos mais simples como peças de Lego.
- **Passagem por Referência:** Como enviar um objeto inteiro para dentro de outro.

📖 A Teoria: O Relacionamento "Tem-Um" (HAS-A)

Pense em um Computador. Ele possui atributos simples como `preco` (double) ou `marca` (String). Mas ele também **TEM UM** `Processador`, **TEM UM** `Teclado` e **TEM UMA** `PlacaMae`. Cada uma dessas partes é, por si só, uma classe complexa.

No FiapRide, a regra de negócio é clara:

- Uma **Viagem** TEM UM **Passageiro**.
- Uma **Viagem** TEM UM **Veiculo**.

No Java, isso significa que o tipo da variável não será `String` ou `int`, será a própria classe que criamos!

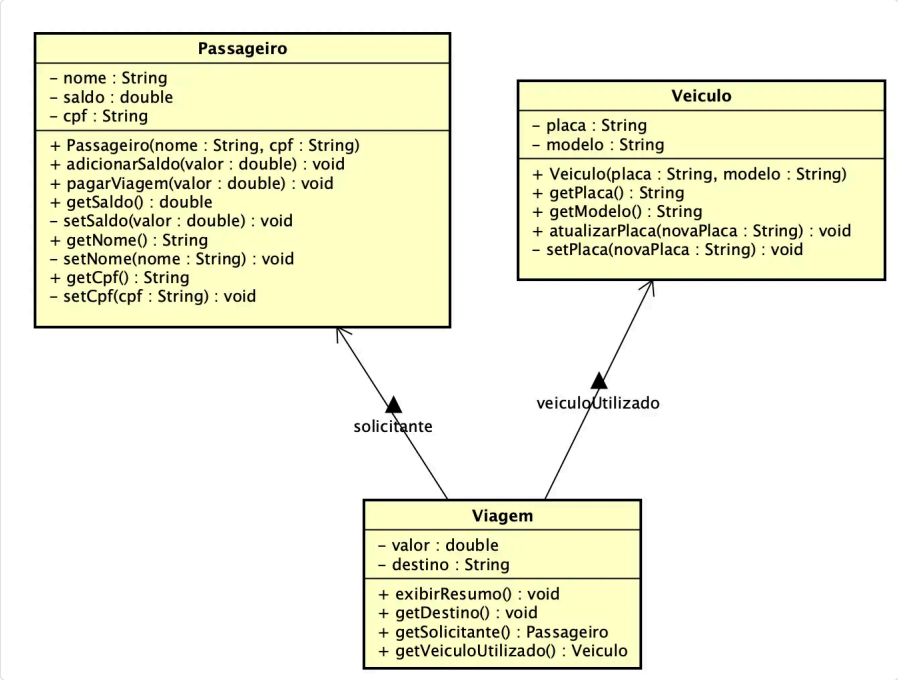
🛠️ Parte 1: Modelagem Visual (Astah)

Vamos desenhar como essas três classes se conectam.

1. Abra o **Astah** no seu projeto FiapRide (onde já estão `Passageiro` e `Veiculo`).
2. Crie uma nova **Class** chamada `Viagem`.
3. Adicione os atributos primitivos básicos:
   - `- valor: double`
   - `- destino: String`

   Adicione os métodos:
   - `+ exibirResumo() : void`
   - `+ getDestino() : void`
   - `+ getSolicitante() : Passageiro`
   - `+ getVeiculoUtilizado() : Veiculo`

4. **O Relacionamento:** Na barra de ferramentas do Astah, selecione a **Association Line** (uma linha reta contínua, geralmente sem seta na ponta ou com uma seta simples de navegação).
5. Clique na classe `Viagem` e arraste a linha até a classe `Passageiro`.
6. Repita o processo: Clique na classe `Viagem` e arraste a linha até a classe `Veiculo`.
7. **Clean Code (Papéis):** Dê um clique na linha de associação do Passageiro e defina papel(nome) dele: `solicitante`. Faça o mesmo com Veiculo, papel(nome) `veiculoUtilizado`.



💻 Parte 2: O Desafio FiapRide (Mão na Massa)

Vamos traduzir esse desenho para o Java.

Passo 1: A Classe `Viagem`

Crie a classe `Viagem` no pacote `br.com.fiapride.model`.

```
package br.com.fiapride.model;

public class Viagem {

    // 1. Atributos Simples
    private String destino;
    private double valor;

    // 2. ATRIBUTOS DE ASSOCIAÇÃO (Objetos Complexos)
    // Clean Code: O nome da variável indica O PAPEL que o objeto desempenha na Viagem!
    private Passageiro solicitante;
    private Veiculo veiculoUtilizado;

    // 3. CONSTRUTOR (Regra de Negócio)
    // Uma viagem SÓ PODE nascer se tiver um destino, um passageiro e um carro!
    public Viagem(String destino, Passageiro solicitante, Veiculo veiculoUtilizado) {
        this.destino = destino;
        this.solicitante = solicitante;
        this.veiculoUtilizado = veiculoUtilizado;
        this.valor = 0.0; // Inicia zerado, pode ser calculado depois
```

```
        System.out.println("✅ Nova viagem solicitada para: " + this.destino);
    }

    // 4. MÉTODOS DE AÇÃO (Comportamento)
    public void exibirResumo() {
        System.out.println("\n--- RESUMO DA VIAGEM ---");
        System.out.println("Destino: " + this.destino);

        // A MÁGICA ACONTECE AQUI: Navegando entre os objetos!
        // Eu entro na Viagem -> pego o Solicitante -> pego o Nome dele.
        System.out.println("Passageiro: " + this.solicitante.getNome());
        System.out.println("Veículo: " + this.veiculoUtilizado.getModelo() + " (Placa: " + this.veic
uloUtilizado.getPlaca() + ")");
        System.out.println("-----------------------\n");
    }

    // Getters para os atributos...
    public String getDestino() { return this.destino; }
    public Passageiro getSolicitante() { return this.solicitante; }
    public Veiculo getVeiculoUtilizado() { return this.veiculoUtilizado; }
}
```

Passo 2: O Teste no Sistema Principal

Como amarramos tudo isso no `main`? Vá até a sua classe `SistemaPrincipal`.

```
package br.com.fiapride.main;
// Importe TODAS as classes necessárias!
import br.com.fiapride.model.Passageiro;
import br.com.fiapride.model.Veiculo;
import br.com.fiapride.model.Viagem;

public class SistemaPrincipal {
    public static void main(String[] args) {

        System.out.println("--- FIAPRIDE: Inicializando Sistema ---");

        // 1. Criamos os "atores" independentes primeiro
        Passageiro ana = new Passageiro("Ana Silva", "222.222.222-22");
        Veiculo carroDoJoao = new Veiculo("ABC-1234", "Toyota Corolla");

        // 2. Criamos a Viagem, conectando (associando) os objetos!
        // Passamos a variável 'ana' e a variável 'carroDoJoao' como parâmetros.
// Saldo ana = 0;
        Viagem viagemDaAna = new Viagem("Avenida Paulista, 1000", ana, carroDoJoao);

        // 3. Testando se os objetos conversam
        viagemDaAna.exibirResumo();

        // 4. Prova da Passagem por Referência:
```

```
        // Se a Ana colocar saldo na conta dela DEPOIS que a viagem foi criada...
        ana.adicionarSaldo(50.0);

        // A viagem "enxerga" esse saldo novo?
        System.out.println("Saldo da Ana consultado ATRAVÉS da Viagem: R$ " + viagemDaAna.getSolicit
ante().getSaldo());

    }
}
```

Se você rodar isso e o resumo da viagem imprimir corretamente os nomes que estão "dentro" dos objetos associados, parabéns! Você acaba de construir uma arquitetura de objetos.

⚡ Microdesafio Pessoal: Conectando seu Mundo

Volte para a classe do **seu** projeto pessoal (Garrafa, Pet, Smartphone, etc). Você percebeu que seu objeto está sozinho?

Sua Missão:

1. Crie uma **NOVA** classe que faça sentido se conectar ao seu objeto atual.
   - *Exemplos:* * Se você tem um `Smartphone`, crie a classe `Bateria` ou a classe `Dono`.
     - Se você tem um `Pet`, crie a classe `Tutor` ou `Coleira`.
     - Se você tem uma `Garrafa`, crie a classe `Tampa` ou `Liquido`.
2. Vá na classe principal do seu projeto e adicione a nova classe como um atributo (crie a Associação).
3. Atualize o Construtor para receber esse novo objeto.
4. Teste no seu `main` imprimindo um dado do objeto associado.

🧠 Checklist: Clean Code (Nomes Baseados em Papéis)

O maior erro de iniciantes ao fazer associações é dar nomes genéricos às variáveis de referência.

- [ ] **Intenção Clara:** Se a classe se chama `Passageiro`, não chame a variável de `passageiro` ou `p` dentro da Viagem. Chame-a pelo papel que ela desempenha ali: `solicitante`, `cliente`, `pagador`.
- [ ] Se a classe é `Veiculo`, não chame de `carro` ou `v`. Chame de `veiculoUtilizado` ou `veiculoDesignado`. Isso torna o código autoexplicativo!

🧠 Pergunta de Reflexão

"No construtor da `Viagem`, nós exigimos o objeto inteiro (`Passageiro solicitante`). Se o nosso resumo só precisa imprimir o nome da pessoa, não seria mais fácil e mais leve pedir apenas a String do nome no construtor da Viagem (`String nomeDoPassageiro`) em vez do objeto todo?"

Pense nas regras de negócio: O que acontece na hora que a Viagem acaba e o sistema precisa descontar o saldo? Se a Viagem tiver apenas a String "Ana Silva", ela consegue mexer no dinheiro dela?

📤 Finalizando (GitHub)

```
git add .
git commit -m "feat: implementa associacao entre Viagem, Passageiro e Veiculo"
git push origin main
```

Bons estudos!

